

Waseet Logistics

A warehouse, a pipeline and a quality suite, built from a folder of CSV files

Project brief and build guide

1. The situation

Waseet Logistics move parcels between ten hubs across Jordan, Lebanon, the UAE, Saudi Arabia and Egypt. Around forty couriers scan a parcel at every point in its journey: picked up, arrived at a hub, departed a hub, out for delivery, delivered, or failed.

Those scans are currently a folder of CSV files that somebody opens in Excel when a customer complains. Nobody can answer "how many Express parcels missed their promised time last month" without a morning of work, and nobody can answer "is our data any good" at all.

You are being asked to build the thing that makes those questions cheap.

This is not Najm

None of the code from Lectures 13 to 16 will run unchanged against this data, and that is deliberate. Every technique you need has been taught in this course. None of the specifics have. Where you reach for an old file, expect to change the columns, the defects and the decisions - not the shape.

What you are building

Six pieces. They are the layers from Lecture 5, in the order the data moves through them.

Layer	What it means here
Ingestion	The daily scan file read as strings with its contract checked first, and the customer dimension pulled from a paginated, rate-limited API
Storage	A Postgres schema with the grain, keys, nullability and index the data actually needs
Processing	Spark over eleven months of history, written to a partitioned Parquet curated zone
Pipeline	extract / check_contract / transform / load - idempotent, logged, with quarantined rows that carry a reason
Orchestration	An Airflow DAG with a sensor, a branch, the right trigger rule and a failure callback
Quality	A suite of rules held as data, a schema drift check against a baseline, and reconciliation

2. What you are given

Everything is in the capstone folder. Nothing outside it is needed, and nothing inside it is decoration.

Path	What it is
data/scans_2026-05-01.csv ... scans_2026-05-21.csv	Three weeks of daily scan files, roughly 1,200 rows a day

Path	What it is
<code>data/scans_history.csv</code>	Eleven months of history - about 285,000 rows, 18 MB. For the Spark part
<code>data/hubs.csv, couriers.csv, service_levels.csv</code>	The lookups. Clean, and already seeded into the warehouse
<code>api/waseet_api.py</code>	The customer API. Paginated, key in a header, rate limited
<code>docker-compose.yml, Dockerfile, init/</code>	Postgres and Airflow on ports 5442 and 8081
<code>pipeline/pipeline.py</code>	Stub. Function signatures and the decisions to make
<code>pipeline/quality.py</code>	Stub. The suite, the drift check, reconciliation
<code>pipeline/ingest_customers.py</code>	Stub. The API pull
<code>dags/waseet_daily.py</code>	Stub. The DAG skeleton, with the sensor already wired
<code>spark/history_job.py</code>	Stub. The historical job

The ports do not clash with Lecture 14's stack. Keep that one running and refer back to it. Read the comments in the stubs: **the questions in them are the assignment**, and they are more specific than this document can be.

The scan file

Nine columns: `scan_id`, `parcel_id`, `customer_id`, `scanned_at`, `hub_id`, `courier_id`, `scan_type`, `weight_kg`, `service_code`.

Some of what you will find in there is deliberate and some of it is not. Working out which is which is part of the job. Three days are worth knowing about in advance, because they are the days your orchestration has to survive rather than your transform.

Day	What happens	What should happen
10 May	No file. It never arrives.	The sensor waits, then times out. The failure names the supplier.
15 May	Eid. The network is closed and the file has a header and no rows.	The branch skips the load. No alert - nothing is wrong.
19 May	The supplier renames a column without telling anyone.	The contract check fails the run loudly, before the transform.

There is a fourth day

One more day is wrong in a way this document is not going to describe. Your reconciliation should find it. If it does not, your reconciliation is decoration.

3. How to build it

Eight phases, in build order. The order is not the order of the layers - it is the order that keeps you from debugging two unknowns at once. Each phase ends in something you can run and check, and nothing in a later phase needs anything a later phase produces.

Two rules run through all of them. **Build the pipeline before the DAG** - a transform debugged through a scheduler is miserable and unnecessary. And **commit small, working states** - if a phase defeats you, hand in the ones that run and say so in the report. A project that is honest about what is missing reads far better than one that is quiet about it.

0

Read the data

You can describe the feed without opening Docker

Open the scan files in pandas, on your laptop, before you start anything else. Nothing in this phase is handed in; everything in it changes decisions you make later.

```
import pandas as pd
day = pd.read_csv("data/scans_2026-05-04.csv", dtype=str)
day.info()
day["scan_type"].value_counts()
```

Look for these, and write down what you find and how many rows it affects:

- `scan_id` values that appear more than once - the same event sent twice
- `scanned_at` in two different formats, roughly 3% of rows in the second one
- `weight_kg` with a comma as the decimal mark, blank, or impossible
- `hub_id` values that are not in `hubs.csv`
- `courier_id` blank - which is a real unassigned scan, not an error
- `scan_type` in mixed case

Done when

You can say, in a sentence each, what is wrong with this feed and roughly how often - with counts, not impressions. Keep these numbers. Section 2 of your report is built from them.

1

Start the stack

Airflow answers on 8081 and the warehouse has four tables

From the `capstone/` folder, with Docker Desktop running:

```
docker compose up -d --build
docker compose logs -f airflow
```

The first build pulls the Airflow image and takes a few minutes. When the log says Airflow is ready, press **Ctrl-C** - that stops watching the log, not the container. Then check both halves:

```
docker compose exec -T airflow airflow version
docker compose exec -T postgres psql -U de -d waseet -c "\dt"
```

The first prints `2.10.5`. The second prints four tables - `hubs`, `couriers`, `service_levels` and `customers` - because the fact table and the run log are yours to write.

What	Where
Airflow UI	<code>http://localhost:8081</code> · admin / admin
Warehouse, from your laptop	<code>localhost:5442</code> · database waseet, user de, password de
Warehouse, from inside a container	<code>postgres:5432</code> · the form <code>docker-compose.yml</code> sets in <code>WASEET_DB_URL</code>

Two addresses, one database

Inside the containers the warehouse is `postgres:5432`. From your laptop it is `localhost:5442`. Both are correct, from different places, and mixing them up is the first thing to check when a connection refuses. `pipeline.py` reads `WASEET_DB_URL` from the environment and falls back to the laptop form, which is the whole trick.

Done when

Both checks above answer. You have not written any code yet, and you should not have.

2

Design the warehouse

parcel_scans and load_log exist, and you can defend their shape

Finish `init/01_create_waseet_db.sql`. The four dimensions are given. The fact table and the run log are yours, and the decisions are in the file as comments. You are not being asked to guess what I want. You are being asked to decide, and to be able to say why in two sentences per decision.

The five decisions

- **Grain.** What does one row of `parcel_scans` mean? One scan event, or something else? A grain you cannot state in one line is the root of most of what goes wrong downstream.
- **Nullability.** A scan with no courier is a real thing in this feed. A scan with no timestamp is not. Making everything `NOT NULL` means you have not read the data; making everything nullable means you have not made a decision.
- **Foreign keys.** A missing hub should not reach this table, but a foreign key is not the only way to stop it, and it is not free.
- **Is `scan_id` a primary key?** Read the note on Najm's `sales` table in Lecture 13 first. The answer there was no, on purpose, and the reason was not laziness. The file contains duplicate scan events, so a key turns a data problem into a crash - which is sometimes what you want.
- **The index.** Which one does the daily load and the daily report actually need? One on the date column is expected. Six with no justification is a conversation about write cost.

Then the run log: one row per run, with what it read, what it loaded and what it rejected. Nothing about the data itself goes in here. Lecture 16 section 5 has the shape.

init/ only runs once

The SQL in `init/` runs on an empty volume and never again. Editing it after the first start does nothing. To apply a change you have to destroy the volume:

```
docker compose down -v then docker compose up -d
```

That deletes everything in the warehouse. You will do it several times while you settle the schema, which is the best argument there is for getting your loads reproducible early.

Done when

`\dt` lists six tables, and you can state the grain of `parcel_scans` in one sentence.

3

Pull the customer dimension

200 customers in the warehouse, and the pull says so

Finish `pipeline/ingest_customers.py`. The API is in `api/waseet_api.py` and starts in-process, so there is nothing to deploy. Run it before the first scan load, because `pipeline.py` checks `customer_id` against this table.

Three things about this API will bite a client that assumes the easy case, and all three were in Lecture 7:

- **The key goes in the X-API-Key header**, not the query string.
- **The answer is an envelope.** The records are under `"customers"`.
- **The first attempt at every third page comes back 429.** Retry the transient and give up on the permanent - a 401 will not become a 200 however many times you ask, and retrying it only delays the moment somebody notices the key is wrong.

Drive the paging loop on `has_more`, not on a page count you worked out yourself. The second one is right until the day the data grows.

The bug this API exists to catch

Two hundred customers arrive over eight pages. **If your pull finishes with fewer than two hundred and does not complain, you have written exactly the bug this API was built to produce.** It is silent, it looks like success, and it is the kind of thing that reaches production.

Done when

`SELECT count(*) FROM customers` returns 200, running the script twice still returns 200, and the second run does not fail.

4

Write the pipeline

One day loads from a terminal, twice, with the same result

Finish `pipeline/pipeline.py` as `extract / check_contract / transform / load`. Run it from a terminal, for one date, until it is right:

```
python pipeline.py --date 2026-05-04
```

What each function owes you

Function	The obligation
<code>extract</code>	Read the day as strings and mean it. Let pandas guess and it decides <code>weight_kg</code> is an object column the moment it meets "12,5" - and then you are checking pandas' opinion of the file instead of the file.
<code>check_contract</code>	Cheap, first, before anything else touches the data. A missing column means nothing downstream can do anything sensible, so there is no reason to find out slowly. One of the twenty-one files renames a column.
<code>transform</code>	Return (good, rejects), and write a reason on every rejected row. Some of the defects are rejects and some are repairs; telling them apart is most of the work. A rejected row with no reason cannot be argued about.
<code>load</code>	Idempotent. Running it twice for the same date must leave the warehouse exactly as running it once did. The 13th is a file that was sent twice - a different problem from the pipeline being run twice, and you need both handled.

Three things that are easy to leave out

- **Log to a file as well as the screen.** The handler is already configured in the stub.
- **Write the run to `load_log` before returning - including on the failure path**, which is the run you will most want a record of.
- **Make `rows_read = rows_loaded + rows_rejected + rows_deduplicated` provable at the end of every run.** If the arithmetic does not close, a row went somewhere you did not decide to send it.

Done when

Run it twice for 4 May and count rows in `parcel_scans` after each. The two numbers are equal. Write them down - the report asks for them.

5

Orchestrate it

Three weeks backfilled, and the awkward days handled

Finish `dags/waseet_daily.py`. The sensor is already wired; the rest is yours. Iterate with `dags test`, which runs the whole DAG in one process and prints straight to your terminal - no unpausing, no waiting for the scheduler:

Task	What it has to get right
<code>wait_for_file</code>	Wait rather than fail on absence. Decide <code>poke_interval</code> , timeout and why mode is reschedule. The 10th never arrives, so the timeout is not hypothetical.
<code>choose</code>	A branch that returns the <code>task_id</code> to run next. The 15th is a header and no rows, and failing that morning is an alert on a day when nothing is wrong.
<code>run_pipeline</code>	The bash call into <code>pipeline.py</code> for the run's date.
<code>skip_day</code>	The other side of the branch. Skipped is not failed.
<code>run_quality</code>	The suite, after the load, against the warehouse.
<code>finish</code>	The join. Its trigger rule is the thing to get right, and it is the one that is silently wrong.

The failure that looks like success

Leave the join at the default `all_success` and every skipped day goes green with the last three tasks quietly not run. There is no red anywhere. Trigger a skip, open the graph, and look at what actually ran - this is the bug you will not notice unless you go looking for it.

Then write `alert`, the failure callback. Write a line an on-call engineer can route from: *which* task failed matters more than that something did. The sensor timing out is the supplier's problem and the load failing is yours, and the two wake up different people.

Then backfill the three weeks. Twice. The row counts must be identical.

Done when

The 10th times out and alerts. The 15th skips, silently, and `finish` still runs. The 19th fails loudly, before the transform. Two backfills of the full three weeks give the same row count.

6

Build the quality suite

A report that says which rule broke and how badly

Finish `pipeline/quality.py`. Rules are data - a list a warehouse manager could review without reading Python, which is the whole reason it is not a set of `.loc` masks inside `transform`.

Three pieces

- **The suite.** Eight to twelve rules, covering at least four of the six dimensions, producing a report that names the rule and counts the rows that broke it. The lab's checks were `not_null`, `unique`, `numeric` and `positive`; the assignment added `date_iso` and `in_set`. You will probably want `in_range` and something for the mixed-case `scan_type`.
- **Schema drift** against a stored baseline. It must catch the 19th. Missing columns and new columns are not the same severity, so return them separately rather than as one verdict.
- **Reconciliation** of file rows against warehouse rows, for every one of the twenty-one days, with every gap explained. A gap is not a failure. An unexplained gap is.

A green suite is not good news

At least one of your rules must actually fail on at least one of the twenty-one files. A suite that passes everything is not evidence that the data is clean; it is an absence of evidence about anything. If yours is green across all twenty-one, the rules are not reaching the data.

Where each check belongs

Four places, and the question that decides between them is what can still be done about it there. A student who puts every check in the transform has learned one place, not four.

Place	What belongs there	What it does on failure
At the boundary	The contract: are the columns the ones we were promised?	Fails the run before a row is processed
During the transform	Validity and consistency, row by row	Quarantines the row with a reason
After the load	Completeness and accuracy at the aggregate level	Fails, or triggers something that can undo
Out of band	Timeliness, on a clock of its own	Warns - and it must not be a task inside the DAG

Why freshness cannot live in the DAG

A freshness check inside `waseet_daily` cannot fire on the morning `waseet_daily` never ran - which is precisely the case it exists for. It belongs in a separate DAG that asks the warehouse how old its newest row is and does not care whether anything else ran.

Finally, decide what the script's exit code should be, and why. **A quality script that always exits 0 cannot be a task in a DAG.**

Done when

The report names a real failure on real data, drift catches the 19th, and your reconciliation table has a row for every one of the twenty-one days.

7

Spark over the history

A partitioned curated zone, and a plan you have read

Finish `spark/history_job.py` and run it on your laptop, not in the container. On Windows, use the environment from Lecture 11 - `JAVA_HOME`, `HADOOP_HOME` and `winutils` - rather than a fresh install.

```
spark-submit history_job.py
```

1. **An explicit schema.** The history file is tidier than the daily ones, so `inferSchema` will not produce nonsense here - it will produce a second full pass over the file to work out what you already know, and a set of types nobody chose.
2. **Read the file with that schema** and `header=True`.
3. **Shape it:** parse `scanned_at`, derive the `scan_date` you will partition on, join `hubs` for region and `service_levels` for promised hours, and drop what the curated zone does not need.
4. **Aggregate** scans per hub per day, and the share of DELIVERED versus FAILED per region per month.
5. **Write the curated zone as Parquet, partitioned by `scan_date`.**

Two questions to answer in the report, and both need you to have looked rather than guessed:

- **Read the plan for at least one join and say what you saw.** `hubs` is ten rows and `service_levels` is four. One of them should be broadcast. Which, and why?
- **How many files did the partitioned write produce**, and is that a reasonable number for 334 days of data? The answer is usually "more than I expected", and noticing is the lesson.

Done when

The curated zone exists on disk, partitioned by date, and you can answer both questions above with a number.

4. What to hand in

A GitHub repository, plus a report. Push your work to GitHub and hand in the link to the repository.

The code

Everything you wrote, in the folder structure it was given to you in. Do not reorganise it - I need to be able to open your version of a file next to mine.

File	What I will look at
<code>init/01_create_waseet_db.sql</code>	The two tables you designed, with your comments on the five decisions
<code>pipeline/pipeline.py</code>	The four functions, the reject reasons, the run log on both paths
<code>pipeline/quality.py</code>	The suite as data, the drift check, the reconciliation, the exit code
<code>pipeline/ingest_customers.py</code>	Retry on transient only, paging on has_more, 200 rows
<code>dags/waseet_daily.py</code>	The branch, the trigger rule on the join, the failure callback
<code>spark/history_job.py</code>	Explicit schema, the two aggregations, the partitioned write
<code>README.md</code>	Yours, not mine - see below

Do not commit the data/ folder, the curated/ output or a virtual environment. A .gitignore with those three lines is the whole of it.

The README

Replace the one you were given. Someone who has never seen this project should be able to start your stack and run one day of data from your README alone. That is a low bar and it is failed often. Four things:

- **How to start it** - the exact commands, in order, including the build.
- **How to run one day** and how to run the backfill.
- **What you did not finish**, stated plainly. This costs you nothing and hiding it costs you a lot.
- **Anything you changed** about the stack itself - a port, a package, a version.

The report

Three to five pages. Five numbered sections, in this order. Prose, not bullet points - if you can only express a decision as a fragment, you have not finished making it. **Screenshots are not evidence unless the number in them is discussed.**

#	Section	What has to be in it
1	Schema and why	Your grain in one sentence, then each of the five decisions with the reason. Where you chose differently from Lecture 13's Najm schema, say so and say why this data is different.
2	What you found in the data	The defects, with counts. Which you repaired, which you rejected, and the rule you used to decide. The fourth bad day, and how you found it.
3	The quality suite	Every rule with its dimension, where it runs, and what happens when it fails. Then the failure it actually caught, on real data, with the count.
4	Reconciliation	A table: one row per day, file rows against warehouse rows, difference, explanation. Every gap explained. This is the section I read first.
5	What I would do next	What you would fix, what you would monitor, and what you would tell Waseet is still unsafe about this pipeline. Half a page is enough if it is specific.

The reconciliation table is the whole project in one page

Twenty-one rows. If the numbers match on every day, one of two things is true: your pipeline is right, or your reconciliation is comparing a number against itself. I can tell which from the explanations column, so write it.

5. What finished looks like

Not a checklist of features - a set of things that are true when you run it. Work towards these rather than towards a line count.

- The stack starts from a clean machine with one command, and the warehouse comes up with six tables.
- A single day loads from a terminal, and loading it a second time changes nothing.
- Rejected rows are somewhere you can query, each with a reason you could show a warehouse manager.
- The full three weeks backfill, twice, to the same row count.
- The 10th alerts, the 15th skips silently, the 19th fails before the transform, and the 13th shows up in reconciliation.
- The quality report names at least one real failure on real data.
- `load_log` has a row for every run, including the ones that failed.
- The curated Parquet zone exists, partitioned by date, and you know how many files it wrote.

If you run out of time

Hand in the phases that work and say so in the README. **A working pipeline with no Spark job beats six half-built layers that will not start**, and it is a much easier conversation. Every phase in section 3 ends in something runnable precisely so that stopping early still leaves you with something whole.

- **Ask.** Anything the stubs do not answer, ask me.